Deep Learning — Complete Structured Syllabus

From Fundamentals to Advanced Topics

Prepared by: Milan Sharma *Reference Books: Goodfellow et al. (MIT Press, 2016) | Nielsen (2015) | Bengio (2009) | Patterson & Gibson (O'Reilly, 2017) | Michelucci (Apress, 2018) | Gulli & Pal (Packt, 2017) | Chollet (Manning, 2017)*

COURSE OVERVIEW

Unit	Topic	Hours
1	Introduction & Foundations	2
2	Deep Network Basics	8
3	Deep Learning Architectures	8
4	Convolutional Neural Networks	7
5	Sequence Modelling — RNNs & Variants	9
6	Autoencoders	7
7	Generative Models	6
8	Transformers & Attention Mechanisms	8
9	Optimizers & Model Evaluation	6
10	Deep Learning Frameworks	6
11	Advanced & Emerging Topics	8
12	Projects & Case Studies	10
Total		**85**

UNIT 1 — INTRODUCTION & FOUNDATIONS

1.1 Course Introduction

- Objectives, scope, and expected outcomes
- How to use reference books and supplementary resources
- Setting up the development environment (Python, pip, conda, GPU setup)

1.2 Artificial Intelligence vs Machine Learning vs Deep Learning

- Definitions and historical perspective
- Rule-based AI vs statistical ML vs representation learning
- Where Deep Learning fits in the AI landscape
- Strengths and limitations of each paradigm

1.3 Why Deep Learning is Important

- Big data and computational advances (GPUs, TPUs, cloud computing)
- Key breakthroughs: ImageNet (2012), AlphaGo, GPT, DALL·E
- Deep Learning vs classical ML: when to choose which
- Current state of the field (2024–2025)

1.4 Practical Applications in Real Life

- Computer Vision: image classification, object detection, segmentation, face recognition
- Natural Language Processing: machine translation, sentiment analysis, chatbots, summarization
- Speech: speech recognition (ASR), text-to-speech (TTS)
- Healthcare: medical imaging, drug discovery, genomics
- Autonomous vehicles and robotics
- Recommendation systems, finance, and scientific discovery

1.5 Challenges in Deep Learning

- Data requirements and data quality
- Computational cost and energy consumption
- Overfitting and generalization
- Explainability and interpretability (Black box problem)

- Bias, fairness, and ethics in AI
- Adversarial attacks and robustness

UNIT 2 — DEEP NETWORK BASICS

2.1 Mathematical Foundations Review

- Linear algebra: vectors, matrices, tensor operations
- Probability: distributions, Bayes' theorem, maximum likelihood estimation (MLE)
- Calculus: derivatives, chain rule, partial derivatives, Jacobian, Hessian
- Information theory: entropy, cross-entropy, KL divergence

2.2 Machine Learning Algorithms (Prerequisites)

- Supervised, unsupervised, and reinforcement learning overview
- Bias-variance tradeoff
- Regularization: L1 (Lasso), L2 (Ridge), Elastic Net
- Cross-validation and model selection

2.3 Building a Machine Learning Algorithm

- Components: dataset, model, loss function, optimizer
- Training, validation, and test split
- Capacity, underfitting, and overfitting
- Hyperparameter tuning strategies

2.4 Neural Networks — Multilayer Perceptron (MLP)

- Biological inspiration: neurons and synapses
- Artificial neuron: weights, bias, weighted sum, activation
- Single-layer perceptron: architecture and limitations (XOR problem)
- Multilayer Perceptron (MLP): hidden layers, depth vs width
- Universal approximation theorem

2.5 Activation Functions

- Sigmoid: formula, properties, vanishing gradient problem
- Tanh: formula, advantages over sigmoid

- ReLU (Rectified Linear Unit): formula, dying ReLU problem
- Leaky ReLU (LReLU): fix for dying ReLU
- Parametric ReLU (PReLU): learnable leakage
- Exponential Linear Unit (ELU / ERELU)
- Scaled Exponential Linear Unit (SELU): self-normalizing networks
- Swish and Mish: modern smooth activations
- GELU (Gaussian Error Linear Unit): used in Transformers/BERT
- Softmax: for multi-class output layer
- Choosing the right activation function: guidelines and best practices

2.6 Feedforward Neural Networks

- Architecture: input → hidden → output layers
- Forward pass: computation graph
- Loss functions: MSE (regression), Binary Cross-Entropy, Categorical Cross-Entropy
- Output units: linear, sigmoid, softmax

2.7 Backpropagation Algorithm

- Intuition: gradient flow through the network
- Chain rule application
- Computing gradients for weights and biases
- Computational graph and automatic differentiation
- Vanishing and exploding gradients — causes and effects
- Gradient clipping

2.8 Stochastic Gradient Descent (SGD) and Variants

- Batch gradient descent
- Stochastic gradient descent (SGD)
- Mini-batch gradient descent
- Learning rate: importance and tuning
- Learning rate schedules: step decay, cosine annealing, warm restarts
- Cyclical learning rates (CLR)

2.9 Curse of Dimensionality

- Problem definition: exponential growth with dimensions
- Implications for deep learning models
- Dimensionality reduction: PCA, t-SNE, UMAP
- Feature selection vs feature extraction

2.10 Deep Feedforward Networks — Practical Aspects

- Weight initialization: Xavier/Glorot, He initialization
- Batch Normalization: motivation, formula, during training vs inference
- Layer Normalization, Group Normalization, Instance Normalization
- Dropout: training vs inference, inverted dropout
- Early stopping

UNIT 3 — DEEP LEARNING ARCHITECTURES OVERVIEW

3.1 Representation Learning

- Hand-engineered features vs learned features
- Hierarchical feature representations
- Transfer learning: pre-trained models, fine-tuning, feature extraction
- Domain adaptation

3.2 Width vs Depth of Neural Networks

- Shallow wide networks vs deep narrow networks
- Universal approximation: shallow vs depth advantage
- Empirical evidence for depth
- Residual connections as a solution to depth challenges

3.3 Unsupervised Pre-training

- Greedy layer-wise pre-training (Bengio et al.)
- Why pre-training helps: initialization, regularization
- From unsupervised pre-training to large-scale self-supervised learning

3.4 Restricted Boltzmann Machines (RBMs)

- Architecture: visible and hidden units
- Energy-based model formulation
- Contrastive divergence training
- Deep Belief Networks (DBNs): stacking RBMs
- Applications and limitations

3.5 Autoencoders (Overview — detailed in Unit 6)

- Encoder–decoder architecture
- Latent space / bottleneck representation
- Reconstruction loss
- Types: undercomplete, overcomplete, regularized

3.6 Taxonomy of Deep Learning Models

- Discriminative vs generative models
- Deterministic vs stochastic models
- Summary table: CNN, RNN, LSTM, GRU, Transformer, AE, VAE, GAN

UNIT 4 — CONVOLUTIONAL NEURAL NETWORKS (CNN)

4.1 Motivation and Intuition

- Problems with fully connected networks for images
- Local connectivity and weight sharing
- Translation invariance

4.2 Core CNN Components

- Convolutional layer: filters/kernels, feature maps, stride, padding (valid vs same)
- Pooling layers: max pooling, average pooling, global average pooling
- Fully connected (dense) layers at the output
- Flattening and global pooling strategies

4.3 Filters and Feature Learning

- Edge detection, texture detection, high-level feature learning
- Visualizing learned filters
- Receptive field

4.4 Regularization in CNNs

- Dropout in CNNs
- Data augmentation: flipping, rotation, cropping, color jitter, mixup, cutout
- Weight decay
- Batch normalization in CNNs

4.5 Popular CNN Architectures

Classic Architectures

- **LeNet-5** (LeCun, 1998) — foundational architecture
- **AlexNet** (Krizhevsky, 2012) — ImageNet breakthrough, ReLU, dropout, GPU training
- **VGGNet** (Simonyan & Zisserman, 2014) — deep with 3×3 filters
- **GoogLeNet / Inception v1** (Szegedy, 2014) — Inception modules, 1×1 convolutions
- **Inception v3, v4** — factorized convolutions, batch norm
- **ResNet** (He, 2015) — residual/skip connections, very deep networks (ResNet-50/101/152)
- **DenseNet** — dense connections, feature reuse

Modern Architectures

- **MobileNet v1/v2/v3** — depthwise separable convolutions, for edge devices
- **EfficientNet** — compound scaling (width, depth, resolution)
- **RegNet** — systematic design space exploration
- **ConvNeXt** — modernized ResNet inspired by Transformers (2022)
- **NFNets** (Normalizer-Free Networks)

4.6 CNN Applications

- Image classification
- Object detection: R-CNN, Fast R-CNN, Faster R-CNN, YOLO (v1–v8), SSD, DETR
- Semantic segmentation: FCN, U-Net, DeepLab

- Instance segmentation: Mask R-CNN
- Face detection and recognition: FaceNet, DeepFace, ArcFace
- Medical image analysis
- Video understanding: 3D CNNs, Two-Stream Networks, SlowFast

4.7 CNN Practical Implementation

- Building CNN in PyTorch and TensorFlow/Keras
- Transfer learning with pre-trained models (ImageNet weights)
- Fine-tuning strategies
- Class Activation Maps (CAM) and Grad-CAM for interpretability

---

UNIT 5 — SEQUENCE MODELLING: RECURRENT AND RECURSIVE NETS

5.1 Introduction to Sequential Data

- Time series, text, audio, video — temporal dependencies
- Why feedforward networks fail for sequences
- Fixed vs variable length inputs/outputs

5.2 Recurrent Neural Networks (RNNs)

- Architecture: hidden state, recurrence equation
- Unrolling through time
- Parameter sharing in time
- Types of RNN tasks: one-to-one, one-to-many, many-to-one, many-to-many
- Vanishing and exploding gradients in RNNs

5.3 Backpropagation Through Time (BPTT)

- Deriving gradients through unrolled network
- Truncated BPTT
- Gradient clipping for exploding gradients

5.4 Bidirectional RNNs

- Architecture: forward + backward pass
- When to use bidirectional RNNs

- Deep bidirectional RNNs

5.5 Long Short-Term Memory (LSTM)

- Motivation: solving vanishing gradient in RNNs
- LSTM cell: cell state and hidden state
- Gates: forget gate, input gate, output gate
- Mathematical formulation
- Peephole connections
- Variants: Depth Gated LSTM, Grid LSTM
- Applications: language modeling, speech recognition, time series

5.6 Gated Recurrent Unit (GRU)

- Architecture: update gate and reset gate
- Comparison with LSTM: fewer parameters
- When to use GRU vs LSTM

5.7 Encoder–Decoder (Seq2Seq) Architecture

- Motivation: variable-length input to variable-length output
- Encoder RNN + Decoder RNN
- Context vector bottleneck problem
- Bahdanau Attention Mechanism
- Applications: machine translation, summarization, question answering

5.8 Recursive Neural Networks

- Tree-structured computation graphs
- Recursive Neural Tensor Networks (RNTN) for sentiment analysis

5.9 Applications of Sequence Models

- Natural Language Processing: text classification, POS tagging, NER
- Speech recognition (ASR): CTC loss, Listen-Attend-Spell
- Music generation, handwriting generation
- Time series forecasting: stock prediction, weather, anomaly detection

---

UNIT 6 — AUTOENCODERS

6.1 Autoencoder Fundamentals

- Encoder–decoder architecture recap
- Undercomplete autoencoders: bottleneck representation
- Reconstruction loss: MSE, binary cross-entropy
- Linear autoencoder vs PCA

6.2 Regularized Autoencoders

- Sparse autoencoders: L1 regularization on activations, KL divergence sparsity
- Denoising autoencoders (DAE): adding noise to inputs, robust representation learning
- Contractive autoencoders (CAE): Frobenius norm of Jacobian penalty

6.3 Stochastic Encoders and Decoders

- Probabilistic formulation
- Variational Autoencoders (VAEs)
    - Latent variable model
    - Reparameterization trick
    - Evidence Lower Bound (ELBO) loss
    - KL divergence term + reconstruction term
    - Generating new samples
    - β-VAE for disentangled representations

6.4 Deep Autoencoders

- Stacking layers for richer representations
- Pre-training deep networks with autoencoders

6.5 Applications of Autoencoders

- Dimensionality reduction and visualization
- Anomaly / outlier detection
- Image denoising and inpainting
- Feature learning for downstream tasks

- Data compression
- Drug discovery and molecular generation

UNIT 7 — GENERATIVE MODELS

7.1 Introduction to Generative Modeling

- Discriminative vs generative models
- Density estimation
- Explicit vs implicit density models
- Evaluation metrics: Inception Score (IS), Fréchet Inception Distance (FID)

7.2 Generative Adversarial Networks (GANs)

- Architecture: Generator and Discriminator
- Minimax game formulation and loss function
- GAN training process and challenges
- Mode collapse, training instability, non-convergence
- Tips for stable GAN training

7.3 GAN Variants

- **DCGAN** (Deep Convolutional GAN) — using CNNs in GANs
- **Conditional GAN (cGAN)** — class-conditioned generation
- **Pix2Pix** — image-to-image translation
- **CycleGAN** — unpaired image-to-image translation
- **StyleGAN / StyleGAN2 / StyleGAN3** — high-fidelity face synthesis
- **Progressive GAN (PGGAN)** — growing the generator and discriminator
- **Wasserstein GAN (WGAN)** — Earth Mover distance, training stability
- **BigGAN** — large-scale class-conditional generation
- **SRGAN** — super-resolution

7.4 Diffusion Models (Modern Generative Models)

- Denoising Diffusion Probabilistic Models (DDPM)
- Forward process: adding noise step by step

- Reverse process: denoising with neural network
- Score-based generative models
- **Latent Diffusion Models (LDMs)** — Stable Diffusion
- **DALL·E 2 / DALL·E 3** — text-to-image
- **Midjourney, Imagen** — overview
- Diffusion vs GANs: quality, diversity, training stability

7.5 Normalizing Flows

- Invertible transformations
- Change of variables formula
- Real NVP, GLOW

7.6 Energy-Based Models (EBMs)

- Contrastive divergence
- Score matching

UNIT 8 — TRANSFORMERS & ATTENTION MECHANISMS

8.1 Attention Mechanism

- Motivation: overcoming seq2seq bottleneck
- Bahdanau (additive) attention
- Luong (multiplicative) attention
- Self-attention: queries, keys, values
- Attention score computation: dot product, scaled dot product

8.2 The Transformer Architecture (Vaswani et al., 2017)

- Encoder stack: multi-head self-attention + feedforward + residual + layer norm
- Decoder stack: masked self-attention + encoder-decoder attention + feedforward
- Positional encoding: sinusoidal and learned
- Multi-head attention: parallelizing attention heads
- Feed-Forward sublayer
- Residual connections and layer normalization

- Full encoder–decoder for seq2seq tasks

8.3 Pre-trained Language Models (NLP)

- **BERT** (Bidirectional Encoder Representations from Transformers)
  - Masked Language Model (MLM) and Next Sentence Prediction (NSP)
  - Fine-tuning for downstream tasks
- **RoBERTa** — improved BERT pre-training
- **ALBERT** — parameter-efficient BERT
- **DistilBERT** — knowledge distillation

8.4 Autoregressive Language Models

- **GPT / GPT-2 / GPT-3** — causal language modeling
- **GPT-4** — multimodal capabilities, reasoning
- Scaling laws for neural language models
- In-context learning and few-shot prompting

8.5 Large Language Models (LLMs) — Modern Landscape

- **LLaMA / LLaMA 2 / LLaMA 3** — open-source LLMs
- **Mistral, Mixtral (MoE)** — efficient inference
- **Claude (Anthropic), Gemini (Google)** — frontier models
- **Instruction tuning and RLHF** (Reinforcement Learning from Human Feedback)
- **Parameter-Efficient Fine-Tuning (PEFT)**: LoRA, QLoRA, Adapter tuning, Prefix tuning
- Quantization: INT8, INT4, GGUF, GPTQ for inference
- Retrieval-Augmented Generation (RAG)
- Prompt engineering, chain-of-thought (CoT) prompting

8.6 Vision Transformers (ViT)

- **ViT** — applying Transformer to image patches
- **DeiT** — data-efficient training
- **Swin Transformer** — hierarchical shifted-window attention
- Hybrid CNN-Transformer models
- **CLIP** — contrastive image-text pre-training (OpenAI)
- **ALIGN, BLIP, BLIP-2** — vision-language models

- Audio: Wav2Vec 2.0, Whisper (OpenAI), HuBERT
- Video: Video Transformer, TimeSformer
- Biology: AlphaFold 2 — protein structure prediction
- Multimodal: Flamingo, GPT-4V, Gemini, LLaVA

UNIT 9 — OPTIMIZERS & MODEL EVALUATION

9.1 Gradient Descent Family

- Vanilla gradient descent: batch, stochastic, mini-batch
- Challenges: saddle points, ravines, ill-conditioning

9.2 Momentum-Based Optimizers

- Classical momentum: exponential moving average of gradients
- Nesterov Accelerated Gradient (NAG)

9.3 Adaptive Learning Rate Optimizers

- **Adagrad** — per-parameter learning rates, accumulates squared gradients, sparse data advantage, diminishing learning rate problem
- **RMSProp** — exponential moving average of squared gradients, fix for Adagrad decay
- **Adam** (Adaptive Moment Estimation) — first and second moment estimates, bias correction, hyperparameters β_1, β_2, ε
- **AdaMax** — infinity norm variant of Adam
- **AMSGrad** — convergence fix for Adam
- **AdamW** — Adam with decoupled weight decay (recommended default)
- **LAMB / LARS** — large-batch training optimizers
- **Lion Optimizer** (2023) — memory-efficient alternative to Adam
- Comparison and guidelines for choosing an optimizer

9.4 Learning Rate Scheduling

- Step decay, exponential decay
- Cosine annealing with warm restarts

- One-cycle policy (Leslie Smith)
- Warmup schedules

9.5 Regularization Techniques

- L1 / L2 weight decay
- Dropout (standard, variational, concrete)
- DropConnect
- Batch Normalization as regularizer
- Data augmentation as implicit regularization
- Label smoothing
- Mixup and CutMix

9.6 Model Evaluation Metrics

- Classification: accuracy, precision, recall, F1-score, ROC-AUC, PR curve
- Regression: MSE, MAE, RMSE, R^2
- Object detection: mAP (mean Average Precision), IoU
- NLP: BLEU, ROUGE, METEOR, perplexity, BERTScore
- Generative models: IS, FID, LPIPS
- Calibration: ECE, reliability diagrams

9.7 Hyperparameter Optimization

- Grid search, random search
- Bayesian optimization
- Hyperband and ASHA
- Neural Architecture Search (NAS): evolutionary, differentiable (DARTS)
- AutoML: AutoKeras, Google AutoML

UNIT 10 — DEEP LEARNING FRAMEWORKS

10.1 TensorFlow & Keras

- TensorFlow architecture: eager execution, computation graphs
- Keras API: Sequential and Functional API, Model subclassing

- tf.data pipeline, tf.keras.callbacks
- TensorBoard for visualization
- TensorFlow Serving and TFLite for deployment
- TensorFlow Extended (TFX) for production ML pipelines

10.2 PyTorch

- Tensors and autograd: dynamic computation graph
- nn.Module: building custom models
- DataLoader and Dataset API
- Custom training loops and nn.functional
- torch.optim: optimizers
- TorchScript and TorchServe for deployment
- PyTorch Lightning: high-level wrapper for structured training
- torchvision, torchaudio, torchtext ecosystem

10.3 JAX (Modern Framework)

- Functional transformations: jit, grad, vmap, pmap
- XLA compilation
- Flax and Haiku: neural network libraries on JAX
- Use in large-scale research (Google, DeepMind)

10.4 HuggingFace Ecosystem

- transformers library: loading and fine-tuning pre-trained models
- datasets library: loading and processing datasets
- Accelerate: multi-GPU/TPU training
- PEFT library: parameter-efficient fine-tuning
- Diffusers library: diffusion models

10.5 Other Frameworks (Overview)

- **Caffe / Caffe2** — historical context, industry use
- **Apache MXNet** — distributed deep learning, AWS SageMaker
- **Theano** — predecessor to modern frameworks, educational value
- **Paddle Paddle** — Baidu's open-source framework

- **MindSpore** — Huawei's framework

10.6 Hardware for Deep Learning

- GPU: NVIDIA CUDA, cuDNN — A100, H100, RTX series
- TPU (Tensor Processing Unit) — Google Cloud
- Edge AI chips: NVIDIA Jetson, Apple Neural Engine, Coral TPU
- Model compression: pruning, quantization, knowledge distillation

---

UNIT 11 — ADVANCED & EMERGING TOPICS

11.1 Self-Supervised Learning

- Contrastive learning: SimCLR, MoCo, BYOL, SimSiam
- Masked autoencoders (MAE) — BERT-style for vision
- DINO / DINOv2 — self-supervised ViT
- Applications in low-label regimes

11.2 Graph Neural Networks (GNNs)

- Graph Convolutional Networks (GCN)
- GraphSAGE, GAT (Graph Attention Networks)
- Message passing framework
- Applications: social networks, drug discovery, recommendation, knowledge graphs

11.3 Reinforcement Learning with Deep Learning

- Deep Q-Network (DQN) — Atari games (DeepMind)
- Policy gradient methods: REINFORCE
- Actor-Critic: A3C, A2C, PPO (Proximal Policy Optimization)
- RLHF (Reinforcement Learning from Human Feedback) — ChatGPT/Claude training
- AlphaGo, AlphaZero, AlphaStar

11.4 Neural Architecture Search (NAS)

- Motivation and search spaces
- Search strategies: evolutionary, reinforcement learning, gradient-based
- DARTS: differentiable architecture search

- EfficientNet and MobileNet as NAS results

11.5 Model Compression and Efficient Deep Learning

- Knowledge distillation: teacher-student training
- Pruning: structured vs unstructured
- Quantization: post-training and quantization-aware training
- Low-rank factorization
- Efficient Transformers: Linformer, Performer, Longformer, Flash Attention

11.6 Explainability & Interpretability (XAI)

- LIME and SHAP for black-box models
- Grad-CAM and Score-CAM for CNNs
- Attention visualization in Transformers
- Mechanistic interpretability: circuits, features, probing classifiers
- Concept-based explanations: TCAV

11.7 Federated Learning & Privacy-Preserving ML

- Federated learning: training on distributed devices
- Differential privacy in deep learning
- Secure aggregation and homomorphic encryption

11.8 Multimodal Deep Learning

- Vision-Language Models (VLMs): CLIP, BLIP, LLaVA
- Audio-Visual learning
- Text-Image generation: DALL·E, Stable Diffusion, Midjourney
- Multimodal LLMs: GPT-4V, Gemini 1.5, Claude 3

11.9 Foundation Models & AI Agents

- What are foundation models?
- Emergent capabilities and scaling laws
- AI agents: ReAct, Toolformer, function calling
- Mixture of Experts (MoE): Mixtral, GPT-4 (rumored), Switch Transformer
- State Space Models (SSMs): Mamba — alternative to Transformers for long sequences

UNIT 12 — DEEP LEARNING PROJECTS & CASE STUDIES

12.1 Computer Vision Projects

- **Cat & Dog Classification** using CNN (PyTorch / Keras)
  - Dataset: Kaggle Dogs vs Cats
  - Data augmentation, transfer learning with ResNet/VGG
  - Metrics: accuracy, confusion matrix
- **Lung Cancer Detection** using CNN
  - Dataset: Luna16 / NIH Chest X-ray
  - Preprocessing CT scans, 3D CNNs, U-Net segmentation
  - Evaluation: sensitivity, specificity, AUC-ROC
- **Object Detection** with YOLOv8 on custom dataset

12.2 Natural Language Processing Projects

- **Sentiment Analysis** with RNN / LSTM / BERT
  - Dataset: IMDb movie reviews, Twitter data
  - Tokenization, word embeddings (GloVe, FastText, BERT embeddings)
  - Evaluation: F1-score, accuracy
- **Text Generation** using LSTM / GPT-2
  - Character-level and word-level language models
  - Temperature sampling and nucleus sampling
- **Named Entity Recognition (NER)** with BERT fine-tuning

12.3 Sequence-to-Sequence Projects

- **Machine Translation** with Transformer (English → French/Spanish)
  - Dataset: WMT / OPUS100
  - Tokenization: BPE, SentencePiece
  - Evaluation: BLEU score
- **Text Summarization** with T5 / BART fine-tuning

12.4 Generative Model Projects

- **Image Generation** with DCGAN (MNIST / CelebA)
- **Image Style Transfer** using feature matching
- **Variational Autoencoder** for latent space exploration and digit generation

12.5 Capstone / End-to-End Project

- Problem definition, dataset selection, preprocessing pipeline
- Model design, training, and evaluation
- Error analysis and iterative improvement
- Deployment: Flask/FastAPI REST API, Gradio/Streamlit demo
- Model monitoring and MLOps concepts

REFERENCE BOOKS

Primary Textbooks

1. **Ian Goodfellow, Yoshua Bengio, Aaron Courville** — *Deep Learning*, MIT Press, 2016 *(The "Deep Learning Bible" — covers theory comprehensively)*
2. **Michael A. Nielsen** — *Neural Networks and Deep Learning*, Determination Press, 2015 *(Best for intuitive understanding of backpropagation and CNNs)*
3. **Yoshua Bengio** — *Learning Deep Architectures for AI*, now Publishers Inc., 2009 *(Foundational reference for representation learning)*
4. **Josh Patterson, Adam Gibson** — *Deep Learning: A Practitioner's Approach*, O'Reilly Media, 2017 *(Hands-on, practitioner-focused, great for DL4J/industry use)*

Secondary References

5. **Umberto Michelucci** — *Applied Deep Learning: A Case-based Approach*, Apress, 2018 *(Case studies from real applications)*
6. **Antonio Gulli, Sujit Pal** — *Deep Learning with Keras*, Packt Publishers, 2017 *(Keras-focused, practical examples)*
7. **François Chollet** — *Deep Learning with Python*, Manning Publications, 2017 *(Best practical intro to Keras/TensorFlow — highly recommended for beginners)*

Additional Modern Resources

8. **Sebastian Raschka** — *Machine Learning with PyTorch and Scikit-Learn*, Packt, 2022

9. **Aurélien Géron** — *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*, O'Reilly, 3rd Ed. 2022
10. **Andrej Karpathy** — *Neural Networks: Zero to Hero* (free video course, highly recommended)

Online Resources

- **Papers With Code** (paperswithcode.com) — latest SOTA benchmarks
- **Hugging Face Docs** (huggingface.co) — pre-trained models and datasets
- **DeepLearning.AI** (deeplearning.ai) — structured specializations by Andrew Ng
- **Fast.ai** (fast.ai) — practical deep learning top-down approach
- **arXiv** (arxiv.org/cs.LG) — latest research papers

SUGGESTED LEARNING PATH

```
BEGINNER (Months 1–2)
  → Unit 1: Foundations & Overview
  → Unit 2: Neural Network Basics + MLP
  → Unit 10: Pick one framework (PyTorch recommended)
  → Unit 12: Simple project (MNIST / sentiment analysis)

INTERMEDIATE (Months 3–5)
  → Unit 4: CNNs + popular architectures
  → Unit 5: RNNs + LSTM + GRU
  → Unit 6: Autoencoders
  → Unit 9: Optimizers + evaluation
  → Unit 12: Computer vision + NLP projects

ADVANCED (Months 6–9)
  → Unit 7: GANs + Diffusion Models
  → Unit 8: Transformers + LLMs + ViTs
  → Unit 3: Representation learning depth
  → Unit 11: GNNs, RL, self-supervised, efficient DL
  → Unit 12: Capstone project + deployment
```

Syllabus Version: 2025 | Covers classical foundations through state-of-the-art topics including LLMs, Diffusion Models, ViTs, Mamba, and more.